

1. Write a short note (4-5 lines) explaining how Kubernetes orchestration helps apps like Zomato or Swiggy handle thousands of food orders at once without downtime.

ANS

- Kubernetes orchestration helps applications like Zomato or Swiggy manage thousands of food orders by running multiple application containers at the same time.
 - It automatically distributes workloads across available pods and can create additional pods when demand increases.
 - If a pod fails, Kubernetes can restart or replace it automatically.
 - This provides scalability, high availability, and helps applications continue running without downtime.
2. Create a Kubernetes Deployment YAML file called spotify-playlist-deployment.yaml that runs two containers: one for a playlist API (use nginx:latest as a placeholder) and one for a song database (use mongo:latest).

ANS

- Create a file named:
 - spotify-playlist-deployment.yaml
- Use the following configuration:
 - apiVersion: apps/v1
 - kind: Deployment
 - metadata:
 - name: spotify-playlist-deployment
 - spec:
 - replicas: 2
 - selector:
 - matchLabels:
 - app: spotify-playlist
 - template:
 - metadata:
 - labels:
 - app: spotify-playlist
 - spec:
 - containers:
 - name: playlist-api

- image: nginx:latest
 - ports:
 - containerPort: 80
 - name: song-database
 - image: mongo:latest
- This Deployment creates **2 pods**, and each pod contains **two containers**:
3. Apply your spotify-playlist-deployment.yaml using kubectl and verify that both containers are running by describing the created pods.

ANS

- Apply the Deployment:
- kubectl apply -f spotify-playlist-deployment.yaml
- Check the Deployment:
- kubectl get deployments
- Check the pods:
- kubectl get pods
- To describe a created pod:
- kubectl describe pod <pod-name>
- For example:
- kubectl describe pod spotify-playlist-deployment-xxxxx
- The pod details should show both containers:
- playlist-api
 - song-database
- To check the status of both containers more clearly:
- kubectl get pods
- Expected result:

NAME	READY	STATUS	RESTARTS	AGE
spotify-playlist-deployment-xxxxx-xxxxx	2/2	Running	0	...
spotify-playlist-deployment-xxxxx-xxxxx	2/2	Running	0	...

2/2 means that **both containers inside the pod are running**.

4. Create a Kubernetes Service YAML file called `spotify-playlist-service.yaml` to expose your playlist API container on port 8080 within the cluster.

Hint: Use type: ClusterIP and targetPort: 80 for the nginx container.

ANS

- Create a file named:
 - `spotify-playlist-service.yaml`
- Use the following configuration:
 - `apiVersion: v1`
 - `kind: Service`
 - `metadata:`
 - `name: spotify-playlist-service`
 - `spec:`
 - `type: ClusterIP`
 - `selector:`
 - `app: spotify-playlist`
 - `ports:`
 - `protocol: TCP`
 - `port: 8080`
 - `targetPort: 80`
 - Apply the Service:
 - `kubectl apply -f spotify-playlist-service.yaml`
- Check the Service:
 - `kubectl get services`
- Expected output:

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
spotify-playlist-service	ClusterIP	10.96.x.x	<none>	8080/TCP

- Here:
 - `port: 8080` → Service port inside the Kubernetes cluster.
 - `targetPort: 80` → Nginx container port.
 - `ClusterIP` → The Service is accessible from inside the Kubernetes cluster.

5. Scale your Deployment to 4 replicas using the `kubectl scale` command and observe how Kubernetes handles the scaling.
Hint: Use `kubectl get pods` to check the number of running pods after scaling.

ANS

- Scale the Deployment from 2 replicas to 4 replicas:
 - `kubectl scale deployment spotify-playlist-deployment --replicas=4`
- Check the Deployment:
 - `kubectl get deployment`
- Check the pods:
 - `kubectl get pods`
- Expected result:

NAME	READY	STATUS	RESTARTS	AGE
spotify-playlist-deployment-xxxxx-xxxxx	2/2	Running	0	...
spotify-playlist-deployment-xxxxx-xxxxx	2/2	Running	0	...
spotify-playlist-deployment-xxxxx-xxxxx	2/2	Running	0	...
spotify-playlist-deployment-xxxxx-xxxxx	2/2	Running	0	...

- After scaling, Kubernetes creates **4 pods**, and each pod contains the two containers: the playlist API and song database.
- Kubernetes manages the additional replicas automatically, helping distribute application workloads and providing better availability.